

**MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA**

**Technical University of Moldova**
**Faculty of Computers, Informatics and Microelectronics**
**Department of Software and Automation Engineering**

**Iulian Pavlov, FAF-241**

# Report

Laboratory work nr. 1

## Classical Encryption Techniques: Caesar Cipher and Caesar Cipher with Permutation

*Checked by:*
**Zaica Maia**, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

# Contents

**1 Theoretical Background** **2**
1.1 The Classical Caesar Cipher . . . 2
1.1.1 Mathematical Formulation . . . 2
1.1.2 Key Space Analysis . . . 2
1.2 Caesar Cipher with Permutation (Keyed Caesar Cipher) . . . 2
1.2.1 Alphabet Permutation Construction . . . 2
1.2.2 Mathematical Formulation . . . 3
1.2.3 Key Space Analysis . . . 3

**2 Implementation Details** **3**
2.1 Alphabet Definition (`Alphabet.cs`) . . . 3
2.2 Input Validation and Logger (`logger.cs`) . . . 4
2.3 Encryption and Decryption Transformations (`CaesarCypher.cs`) . . . 5
2.4 Building the Permuted Alphabet (`task2.cs`) . . . 6

**3 Results and Verification** **6**
3.1 Task 1: Standard Caesar Cipher . . . 6
3.1.1 Test Case Specification and Mathematical Verification . . . 6
3.1.2 Program Console Executions . . . 6
3.2 Task 2: Caesar Cipher with Permutation . . . 7
3.2.1 Test Case Specification and Alphabet Permutation . . . 7
3.2.2 Program Console Executions . . . 8
3.3 Input Validation and Error Handling Tests . . . 8

**4 Conclusions** **9**

**5 GitHub Repository** **9**

# 1 Theoretical Background

## 1.1 The Classical Caesar Cipher

The Caesar cipher is one of the oldest symmetric cryptosystems, categorized as a **monoalphabetic substitution cipher**. In this scheme, every letter in the plaintext is shifted by a fixed integer key $k$ across an alphabet $\mathcal{A}$ of cardinality $N = |\mathcal{A}|$.

### 1.1.1 Mathematical Formulation

Let the alphabet symbols be assigned discrete numerical positions in the residue ring $\mathbb{Z}_N = \{0, 1, \ldots, N-1\}$.

- **Encryption**:
$$E(p, k) = (p + k) \pmod{N} \tag{1}$$
where $p \in \mathbb{Z}_N$ is the numerical value of the plaintext letter, $k$ is the integer key, and $N$ is the alphabet length.

- **Decryption**:
$$D(c, k) = (c - k + N) \pmod{N} \tag{2}$$
where $c \in \mathbb{Z}_N$ represents the ciphertext character index.

### 1.1.2 Key Space Analysis

For the Romanian alphabet containing $N = 31$ characters (`A, Ă, Â, B, C, D, E, F, G, H, I, Î, J, K, L, M, N, O, P, Q, R, S, Ş, T, Ţ, U, V, W, X, Y, Z`):

- The valid non-trivial key space is constrained by:
$$\mathcal{K} = \{k \in \mathbb{Z} \mid 0 < k < 30\} \implies |\mathcal{K}| = 29 \tag{3}$$

- Because $|\mathcal{K}| = 29$ is extraordinarily small, an attacker can exhaust all candidate keys almost instantaneously via a brute-force ciphertext-only attack.

## 1.2 Caesar Cipher with Permutation (Keyed Caesar Cipher)

To strengthen the classical Caesar cipher against simple inspection, a second secret key—a keyword $k_2$—is utilized to permute the alphabet before applying a numerical shift $k_1$.

### 1.2.1 Alphabet Permutation Construction

Given the standard alphabet $\mathcal{A}$ and a secret keyword $k_2$ (with $|k_2| \geq 7$ containing only alphabet letters):

1. Convert $k_2$ to uppercase.
2. Remove duplicate characters within $k_2$, preserving only their first occurrence.
3. Append all remaining letters of the original alphabet $\mathcal{A}$ in their original order.

Formally, the permuted alphabet $\mathcal{A}'$ is:

$$\mathcal{A}' = \text{Distinct}(k_2 \parallel \mathcal{A}) \tag{4}$$

where $\parallel$ denotes concatenation.

### 1.2.2 Mathematical Formulation

Let $\text{index}_{\mathcal{A}'}(x)$ denote the 0-based index of character $x$ in $\mathcal{A}'$:

- **Encryption**:
$$c = \mathcal{A}'\left[(\text{index}_{\mathcal{A}'}(p) + k_1) \pmod N\right] \tag{5}$$
- **Decryption**:
$$p = \mathcal{A}'\left[(\text{index}_{\mathcal{A}'}(c) - k_1 + N) \pmod N\right] \tag{6}$$

### 1.2.3 Key Space Analysis

The key consists of the pair $(k_1, k_2)$:

- Shift key $k_1 \in \{1, 2, \ldots, 29\}$ $(0 < k_1 < 30)$.
- Keyword $k_2$: With length $|k_2| \geq 7$ chosen from $N = 31$ alphabet characters without repetition, the number of distinct permutations induced solely by keyword prefixes of length $L = 7$ is:
$$P(31, 7) = \frac{31!}{(31-7)!} = 31 \times 30 \times 29 \times 28 \times 27 \times 26 \times 25 \approx 1.32 \times 10^{10} \tag{7}$$
Factoring in $k_1$, the composite key space expands to:
$$|\mathcal{K}_{\text{total}}| \approx 29 \times 1.32 \times 10^{10} \approx 3.84 \times 10^{11} \tag{8}$$
This eliminates trivial brute-force recovery. However, being a monoalphabetic substitution, it remains susceptible to frequency analysis.

# 2 Implementation Details

The laboratory tasks were implemented in C# (.NET 10) organized into modular classes: alphabet definitions, rigorous input validation and logging, mathematical cipher transformations, and task execution pipelines.

## 2.1 Alphabet Definition (`Alphabet.cs`)

The Romanian alphabet consists of 31 characters, accommodating diacritics (`Ă, Â, Î, Ş, Ţ`). Both Romanian and English alphabets are stored as static arrays.

```
1  namespace Lab1;
2
3  public static class Alphabet
4  {
5      public static readonly char[] RomanianAlphabet =
6      {
7          'A', 'Ă', 'Â', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'Î', 'J', 'K', 'L', 'M
       ',
8          'N', 'O', 'P', 'Q', 'R', 'S', 'Ş', 'Ţ', 'U', 'V', 'W', 'X', 'Y', 'Z'
9      };
10     public static readonly char[] EnglishAlphabet =
11     {
12         'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M',
```

```
        'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z'
    };
}
```

Listing 1: Romanian and English alphabets defined in Alphabet.cs

*Comment:* Storing explicit character arrays guarantees deterministic indexing and consistent uppercase manipulation across platforms.

## 2.2 Input Validation and Logger (`logger.cs`)

To fulfill the requirements ($0 < k_1 < 30$, text composed strictly of alphabet characters, and keyword $k_2$ having at least 7 alphabet characters), a centralized input validator was built.

```
namespace Lab1;

public static class Logger
{
    public static bool IsValidKey(int key) => key > 0 && key < 30;

    public static bool IsLetterInAlphabet(char c, char[] alphabet) =>
        Array.IndexOf(alphabet, char.ToUpper(c)) != -1;

    public static bool ValidateText(string text, char[] alphabet, out char invalidChar
    )
    {
        invalidChar = '\0';
        if (string.IsNullOrEmpty(text)) {
            Console.WriteLine("[Error] Text cannot be empty.");
            return false;
        }
        foreach (char c in text) {
            if (!IsLetterInAlphabet(c, alphabet)) {
                invalidChar = c;
                Console.WriteLine($"[Error] '{c}' is not a valid letter in the
    alphabet.");
                return false;
            }
        }
        return true;
    }

    public static bool ValidateKey2(string key2, char[] alphabet)
    {
        if (string.IsNullOrEmpty(key2) || key2.Length < 7) {
            Console.WriteLine("[Error] Key 2 must be at least 7 characters long.");
            return false;
        }
        foreach (char c in key2) {
            if (!IsLetterInAlphabet(c, alphabet)) {
                Console.WriteLine($"[Error] Key 2 contains '{c}', which is not a valid
     letter.");
                return false;
            }
        }
        return true;
    }
```

```
41
42      public static int ReadKey(string prompt = "Enter the key (0 < key < 30): ")
43      {
44          while (true) {
45              Console.Write(prompt);
46              if (!int.TryParse(Console.ReadLine(), out int key) || !IsValidKey(key)) {
47                  Console.WriteLine("[Error] Key must satisfy 0 < key < 30 (between 1
    and 29).");
48                  continue;
49              }
50              return key;
51          }
52      }
53  }
```

Listing 2: Input validation routines in logger.cs

*Comment:* Interactive prompt loops provide clear error feedback upon invalid entry without program termination.

## 2.3 Encryption and Decryption Transformations (`CaesarCypher.cs`)

Modular arithmetic implements shifts in both directions, compensating for negative modulo results.

```
1  namespace Lab1;
2
3  public static class CaesarCypher
4  {
5      public static char Encryption(char c, int key, char[] alphabet)
6      {
7          int index = Array.IndexOf(alphabet, c);
8          if (index == -1) return c;
9
10         int shift = (index + key) % alphabet.Length;
11         if (shift < 0) shift += alphabet.Length;
12
13         return alphabet[shift];
14     }
15
16     public static char Decryption(char c, int key, char[] alphabet)
17     {
18         int index = Array.IndexOf(alphabet, c);
19         if (index == -1) return c;
20
21         int shift = (index - key) % alphabet.Length;
22         if (shift < 0) shift += alphabet.Length;
23
24         return alphabet[shift];
25     }
26 }
```

Listing 3: Encryption and Decryption functions in CaesarCypher.cs

*Comment:* By receiving the `alphabet` array as an argument, the exact same implementation serves both Task 1 (standard alphabet) and Task 2 (permuted alphabet).

### 2.4 Building the Permuted Alphabet (`task2.cs`)

Task 2 dynamically derives the permuted alphabet using LINQ `Concat` and `Distinct`.

```
char[] permutedAlphabet = key2.Concat(Alphabet.RomanianAlphabet)
                              .Distinct()
                              .ToArray();
```

Listing 4: Alphabet permutation logic in task2.cs

*Comment:* `Distinct()` retains only the first occurrence of each character in $k_2 \parallel \mathcal{A}$, generating a valid bijective permutation of the alphabet.

## 3 Results and Verification

To rigorously test and verify the implemented algorithms, comprehensive test cases were selected, reflecting authentic examples from classical cryptography curricula. Both the mathematical step-by-step transformations and the actual program executions are presented below.

### 3.1 Task 1: Standard Caesar Cipher

#### 3.1.1 Test Case Specification and Mathematical Verification

- **Alphabet**: Romanian Alphabet ($N = 31$ characters)
- **Secret Key**: $k = 5$ (satisfies $0 < k < 30$)
- **Plaintext**: `SECURITATE`

Table 1: Mathematical step-by-step verification of Task 1 encryption and decryption

| **Plaintext** ($p$) | **Index** ($p$) | $(p+5) \pmod{31}$ | **Ciphertext** ($c$) | $(c-5) \pmod{31}$ | **Decrypted** |
|---|---|---|---|---|---|
| S | 21 | $(21+5) = 26$ | W | $(26-5) = 21$ | S |
| E | 6 | $(6+5) = 11$ | Î | $(11-5) = 6$ | E |
| C | 4 | $(4+5) = 9$ | H | $(9-5) = 4$ | C |
| U | 25 | $(25+5) = 30$ | Z | $(30-5) = 25$ | U |
| R | 20 | $(20+5) = 25$ | U | $(25-5) = 20$ | R |
| I | 10 | $(10+5) = 15$ | M | $(15-5) = 10$ | I |
| T | 23 | $(23+5) = 28$ | X | $(28-5) = 23$ | T |
| A | 0 | $(0+5) = 5$ | D | $(5-5) = 0$ | A |
| T | 23 | $(23+5) = 28$ | X | $(28-5) = 23$ | T |
| E | 6 | $(6+5) = 11$ | Î | $(11-5) = 6$ | E |

**Theoretical Result:** Plaintext `SECURITATE` maps to ciphertext `WÎHZUMXDXÎ`, and deciphers precisely back to `SECURITATE`.

#### 3.1.2 Program Console Executions

The program was executed for both encryption and decryption workflows:

Listing 5: Task 1 Encryption Execution

```
Enter the key (0 < key < 30): 5
Enter the text: SECURITATE
1. Encrypt text
2. Decrypt text
Enter your choice: 1
Result: WÎHZUMXDXÎ
```

Listing 6: Task 1 Decryption Execution

```
Enter the key (0 < key < 30): 5
Enter the text: WÎHZUMXDXÎ
1. Encrypt text
2. Decrypt text
Enter your choice: 2
Result: SECURITATE
```

## 3.2 Task 2: Caesar Cipher with Permutation

### 3.2.1 Test Case Specification and Alphabet Permutation

- **Shift key ($k_1$)**: 7 (satisfies $0 < k_1 < 30$)
- **Keyword ($k_2$)**: `ALGORITM` (length $8 \geq 7$, consisting only of alphabet characters)
- **Plaintext**: `INFORMATICA`

**Permuted Alphabet Construction ($\mathcal{A}'$):**

1. Unique keyword characters: `A, L, G, O, R, I, T, M` (8 characters).
2. Remaining alphabet characters appended in order:
   `Ă, Â, B, C, D, E, F, H, Î, J, K, N, P, Q, S, Ş, Ţ, U, V, W, X, Y, Z` (23 characters).
3. Total cardinality: $8 + 23 = 31$ characters.

Table 2: Mathematical step-by-step verification of Task 2 encryption and decryption

| **Plaintext ($p$)** | **Index in $\mathcal{A}'$** | $(i + 7) \pmod{31}$ | **Ciphertext ($c$)** | $(i_c - 7) \pmod{31}$ | **Decrypted** |
|---|---|---|---|---|---|
| I | 5 | $(5 + 7) = 12$ | D | $(12 - 7) = 5$ | I |
| N | 19 | $(19 + 7) = 26$ | V | $(26 - 7) = 19$ | N |
| F | 14 | $(14 + 7) = 21$ | Q | $(21 - 7) = 14$ | F |
| O | 3 | $(3 + 7) = 10$ | B | $(10 - 7) = 3$ | O |
| R | 4 | $(4 + 7) = 11$ | C | $(11 - 7) = 4$ | R |
| M | 7 | $(7 + 7) = 14$ | F | $(14 - 7) = 7$ | M |
| A | 0 | $(0 + 7) = 7$ | M | $(7 - 7) = 0$ | A |
| T | 6 | $(6 + 7) = 13$ | E | $(13 - 7) = 6$ | T |
| I | 5 | $(5 + 7) = 12$ | D | $(12 - 7) = 5$ | I |
| C | 11 | $(11 + 7) = 18$ | K | $(18 - 7) = 11$ | C |
| A | 0 | $(0 + 7) = 7$ | M | $(7 - 7) = 0$ | A |

**Theoretical Result:** Plaintext `INFORMATICA` encrypts to `DVQBCFMEDKM` and deciphers symmetrically back to `INFORMATICA`.

### 3.2.2 Program Console Executions

Listing 7: Task 2 Encryption Execution

```
Enter the key (0 < key < 30): 7
Enter the text: INFORMATICA
Enter the second key (at least 7 letters): ALGORITM
Permuted alphabet: A, L, G, O, R, I, T, M, Ă, Â, B, C, D, E, F, H, Î, J, K, N, P, Q, S
    , Ş, Ţ, U, V, W, X, Y, Z
1. Encrypt text
2. Decrypt text
Enter your choice: 1
Result: DVQBCFMEDKM
```

Listing 8: Task 2 Decryption Execution

```
Enter the key (0 < key < 30): 7
Enter the text: DVQBCFMEDKM
Enter the second key (at least 7 letters): ALGORITM
Permuted alphabet: A, L, G, O, R, I, T, M, Ă, Â, B, C, D, E, F, H, Î, J, K, N, P, Q, S
    , Ş, Ţ, U, V, W, X, Y, Z
1. Encrypt text
2. Decrypt text
Enter your choice: 2
Result: INFORMATICA
```

## 3.3 Input Validation and Error Handling Tests

The implementation enforces the following validation checks to guarantee system reliability:

1. **Numerical key bounds** ($0 < k < 30$): Negative numbers, zero, and values $\geq 30$ are rejected.
2. **Plaintext alphabet conformance**: Spaces, numeric digits, and punctuation marks are detected and rejected.
3. **Keyword length requirement** ($|k_2| \geq 7$): Keywords with fewer than 7 characters are rejected.

Listing 9: Input Validation Demonstration in Console

```
Enter the key (0 < key < 30): -5
[Error] Key must satisfy 0 < key < 30 (between 1 and 29). You entered: -5
Enter the key (0 < key < 30): 30
[Error] Key must satisfy 0 < key < 30 (between 1 and 29). You entered: 30
Enter the key (0 < key < 30): 42
[Error] Key must satisfy 0 < key < 30 (between 1 and 29). You entered: 42
Enter the key (0 < key < 30): 7
Enter the text: HELLO WORLD 2026!
[Error] ' ' is not a valid letter in the alphabet.
Enter the text: INFORMATICA
Enter the second key (at least 7 letters): CIPHER
[Error] Key 2 must be at least 7 characters long.
Enter the second key (at least 7 letters): UTM
[Error] Key 2 must be at least 7 characters long.
```

```
Enter the second key (at least 7 letters): ALGORITM
Permuted alphabet: A, L, G, O, R, I, T, M, Ă, Â, B, C, D, E, F, H, Î, J, K, N, P, Q, S
    , Ş, Ţ, U, V, W, X, Y, Z
1. Encrypt text
2. Decrypt text
Enter your choice: 1
Result: DVQBCFMEDKM
```

# 4 Conclusions

In this laboratory work, the classical Caesar cipher and the keyed Caesar cipher with alphabet permutation were studied, implemented, and verified in C#. While the classical Caesar cipher provides minimal security due to its small key space of only 29 non-trivial keys ($0 < k < 30$) which can be broken via brute force in milliseconds, the introduction of a keyword permutation ($k_2 \geq 7$) expands the key space to over $3.8 \times 10^{11}$ combinations, effectively preventing manual brute-force attacks. However, because both methods preserve a one-to-one character mapping, they remain inherently vulnerable to frequency analysis attack on natural language statistics. Key software engineering practices, such as safe input validation, structured error logging, and robust modular arithmetic, were successfully demonstrated throughout the implementation.

# 5 GitHub Repository

The complete source code, project configuration, and documentation for this laboratory work are version-controlled and hosted on GitHub.

- **Repository URL**: https://github.com/PavlovIulian/CryptographyAndSecurity
- **Project**: Cryptography and Security – Laboratory Work 1
- **Author**: Iulian Pavlov (FAF-241)